CI/CD Pipeline Integration: Streamlining Development with Jenkins, Docker, and Kubernetes Tools.

Deployment of 3 Tier Architecture in AWS with CI/CD pipeline

In today's fast-paced software development landscape, integrating a Continuous Integration and Continuous Deployment (CI/CD) pipeline is essential for delivering high-quality applications efficiently. This blog post will discuss how we implemented a CI/CD pipeline using Jenkins, Docker, and Kubernetes, focusing on automating build, testing, and deployment processes for a microservices-based application.

What is CI/CD?

CI/CD stands for Continuous Integration and Continuous Deployment, two practices that streamline software development and delivery:

****Continuous Integration (CI)**** involves regularly merging code changes into a shared repository, allowing for automated builds and tests to identify bugs early in the development process.
****Continuous Deployment (CD)**** automates the release of code changes to production, ensuring that the latest version of the application is always available to users.

Software and Tools that I used ➖

Jenkins

Jenkins is a widely used open-source automation server that provides a robust framework for implementing CI/CD pipelines. It supports integration with various tools and plugins, making it a versatile choice for automation.

Docker

Docker is a containerization platform that enables developers to package applications and their dependencies into lightweight, portable containers. This ensures consistency across different environments, from development to production.

Kubernetes

Kubernetes is a powerful orchestration platform for managing containerized applications at scale. It automates the deployment, scaling, and management of containerized applications, providing features like load balancing, self-healing, and rolling updates.

Our CI/CD Pipeline Implementation

Deploy the EC2 instances with an Apache Web Service in public.
To improve the performance, an application load balancer (ALB) and Auto Scaling Group (ASG) and (RDS).

1. Deploy and EC2 instance (USER-DATA)
Here we are deploying a simple WordPress website on an Apache web server on aws EC2 instance

2 . Here we deploy code deploy agent in EC2

Script for automation

```
#!/bin/bash
sudo apt-get update -y
sudo apt install apache2 -y
sudo systemctl enable apache2
sudo apt install mysql-server -y
sudo apt-get install mysql-server mysql-client -y
sudo systemctl enable mysql
sudo apt install php libapache2-mod-php php-mysql -y
sudo apt install php-curl php-gd php-mbstring php-xml php-xmlrpc php-soap php-intl php-zip -y
sudo a2enmod rewrite
sudo systemctl restart apache2
wget https://wordpress.org/latest.tar.gz
tar -xzf latest.tar.gz
rm -rf latest.tar.gz
cp -r wordpress/* /var/www/html/
cd /var/www/html/
sudo rm -rf index.html/
sudo mv wp-config-sample.php wp-config.php
sudo chmod -R 755 wp-content
```

sudo chown -R ubuntu:www-data wordpress
sudo service apache2 restart

sudo apt-get update -y
sudo apt install wget ruby -y
wget
https://aws-codedeploy-us-east-1.s3.us-east-1.amazonaws.com/latest/install
chmod +x ./install
sudo ./install auto
sudo apt-get update -y
sudo service codedeploy-agent start
sudo service codedeploy-agent status
sudo service apache2 restart

==========

1. Setting Up Jenkins

We began by setting up Jenkins on a dedicated server. This involved:

- Installing Jenkins and configuring the necessary plugins for Docker and Kubernetes integration.
- Setting up Jenkins credentials to securely access our version control system and container registry.

2. Creating the Pipeline

Next, we defined a Jenkins pipeline using a `Jenkinsfile`, which outlined the stages of our CI/CD process:

****Build Stage****: In this stage, Jenkins retrieves the latest code from our version control system (e.g., Git). It then uses Docker to build the application image based on the Dockerfile, ensuring that all dependencies are included.

****Test Stage**:** After the build is successful, Jenkins runs automated tests (unit tests, integration tests) inside a Docker container to validate the application. This step helps catch any issues early, improving code quality.

****Push Stage**:** If all tests pass, the application image is pushed to a container registry (like Docker Hub or a private registry). This makes the image available for deployment.

Deploy Stage: Finally, Jenkins triggers a Kubernetes deployment. Using `kubectl`, the pipeline updates the application in the Kubernetes cluster with the new image. Kubernetes handles the rollout, ensuring zero downtime and managing replicas for load balancing.

3. Automating the Pipeline

To enhance automation, we configured webhooks in our version control system to trigger the Jenkins pipeline automatically whenever code is pushed. This real-time integration allows for faster feedback and reduces manual intervention.

Benefits of Our CI/CD Pipeline

The implementation of the CI/CD pipeline for our microservices-based application has led to significant improvements in development efficiency:

- **Faster Feedback Loop**: Automated testing and integration enable developers to receive immediate feedback on code changes, reducing the time spent on debugging.
- **Consistency Across Environments**: Docker containers ensure that applications run the same way in development, testing, and production environments, minimizing the "it works on my machine" syndrome.
- **Reduced Deployment Time**: Automated deployments through Kubernetes have drastically reduced the time it takes to get new features into production, enhancing our ability to respond to user needs quickly.
- **Improved Collaboration**: The CI/CD process promotes collaboration among team members, as everyone can rely on the same automated processes and tools.

Conclusion

Implementing a CI/CD pipeline using Jenkins, Docker, and Kubernetes has transformed our development process. By automating build, testing, and deployment for our microservices-based application, we've improved efficiency, enhanced collaboration, and ensured the delivery of high-quality software. As the software development landscape continues to evolve, embracing CI/CD practices will be key to staying competitive and delivering value to users.

Whether you're just starting with CI/CD or looking to enhance your existing pipeline, consider leveraging these powerful tools to streamline your development workflow.